Advanced Features Documentation

Profitability Pack v3.0

Spis treści

1. Kelly Criterion Position Sizing
2. Scale Out (Partial Profit Taking)
3. Smart Order Execution
4. Dodawanie nowych par
5. API Endpoints

Kelly Criterion Position Sizing

Opis

Kelly Criterion to matematyczna formuła do obliczania optymalnej wielkości pozycji bazując na historical win rate i średnim R:R.

Formula: `Kelly = (p * b - q) / b`
- p = probability of winning (win_rate)
- q = probability of losing (1 - p)
- b = ratio of avg_win / avg_loss

Funkcje

KellyCalculator

```
from risk_manager import KellyCalculator

kelly = KellyCalculator(config)

# Oblicz Kelly fraction
result = kelly.calculate_kelly_fraction(trades_history)
print(f"Optimal position: {result.optimal_position_pct}%")

# Oblicz wielkość pozycji
quantity, risk_pct = kelly.get_optimal_position_size(
    capital=10000,
    entry_price=50000,
    stop_loss=49000,
    direction='LONG'
)
```

Konfiguracja

```yaml
position_sizing:
  method: "kelly"  # lub "fixed"
  kelly:
    enabled: true
    use_half_kelly: true  # 50% Kelly dla bezpieczeństwa
    min_trades_required: 20  # Min trade'ów do kalkulacji
    update_frequency: 50  # Aktualizacja co N trade'ów
    min_position_pct: 0.5  # Min 0.5% kapitału
    max_position_pct: 5.0  # Max 5% kapitału
  fixed:
    risk_per_trade: 2.0  # Fallback fixed %
```

Bezpieczeństwo

1. **Half-Kelly**: Domyślnie używamy połowy obliczonego Kelly dla zmniejszenia ryzyka
2. **Min/Max Limits**: Position size ograniczone do 0.5% - 5% kapitału
3. **Fallback**: Jeśli <20 trade'ów, używamy fixed 2%
4. **Confidence Levels**: LOW (<30 trades), MEDIUM (30-50), HIGH (>50)

Scale Out (Partial Profit Taking)

Opis

Zamiast zamykać całą pozycję na jednym TP, zamykamy częściami na różnych poziomach profit:

- **TP1**: 1.0x R:R → Zamknij 50% → Move SL to breakeven
- **TP2**: 1.5x R:R → Zamknij 30% → Move SL to TP1
- **TP3**: 2.5x R:R → Zamknij 20% → Trailing stop

Funkcje

ScaleOutManager

```python
from execution_layer import ScaleOutManager

scale_out = ScaleOutManager(config)

# Rejestruj pozycję
state = scale_out.register_position(
    symbol='BTCUSDC',
    entry_price=50000,
    stop_loss=49000,
    quantity=1.0,
    direction='LONG'
)

# Sprawdź triggery
triggered = scale_out.check_scale_out_triggers('BTCUSDC', current_price=51000)

# Wykonaj partial close
for level in triggered:
    closed_qty, pnl = scale_out.execute_partial_close('BTCUSDC', level, 51000)
    new_sl = scale_out.update_stop_loss_after_tp('BTCUSDC', level.level_id)
```

Konfiguracja

```yaml
profit_taking:
  scale_out:
    enabled: true
    levels:
      tp1:
        rr_multiplier: 1.0
        close_percentage: 50
      tp2:
        rr_multiplier: 1.5
        close_percentage: 30
      tp3:
        rr_multiplier: 2.5
        close_percentage: 20
    move_sl_to_breakeven_after_tp1: true
    move_sl_to_tp1_after_tp2: true
```

Przykład Scenario

```
Entry: $50,000 | SL: $49,000 (Risk = $1,000)

TP1 @ $51,000 (1.0R): Zamknij 50% → SL → $50,000 (breakeven)
TP2 @ $51,500 (1.5R): Zamknij 30% → SL → $51,000 (TP1)
TP3 @ $52,500 (2.5R): Zamknij 20% → Trailing stop

Wynik jeśli wszystkie TP triggered:
- TP1: 50% * $1,000 = $500
- TP2: 30% * $1,500 = $450
- TP3: 20% * $2,500 = $500
Total: $1,450 (vs $2,500 all-or-nothing przy 100% TP3)
```

Smart Order Execution

Opis

Optymalizacja kosztów wykonania przez preferowanie LIMIT orders (maker) zamiast MARKET (taker).

Fee rates:
- Regular Maker: -0.01% (rebate!)
- Regular Taker: +0.04%
- VIP1 Taker: +0.03%
- VIP2 Taker: +0.02%

Funkcje

SmartOrderExecution

```python
from execution_layer import SmartOrderExecution

smart_exec = SmartOrderExecution(config)

# Sprawdź spread
is_ok, spread_pct = smart_exec.check_spread(bid=50000, ask=50040)

# Szacuj koszty
cost = smart_exec.estimate_total_cost(quantity=0.1, price=50000, order_type='maker')
print(f"Fee: ${cost['fee']}, Slippage: ${cost['estimated_slippage']}")

# Wykonaj smart order
order, order_type = executor.execute_order_smart(
    symbol='BTCUSDC',
    side='BUY',
    quantity=0.1,
    expected_price=50000,
    bid=50000,
    ask=50040
)
```

Konfiguracja

```yaml
execution:
  prefer_maker_orders: true
  limit_order_timeout: 30  # seconds
  max_spread_pct: 0.1  # Skip trade if spread > 0.1%
  track_slippage: true
  vip_tier: 0  # 0=regular, 1=VIP1, 2=VIP2
```

Strategia Wykonania

1. Sprawdź spread - jeśli > 0.1% → **SKIP TRADE**
2. Place LIMIT order na best bid (BUY) / best ask (SELL)
3. Czekaj max 30 sekund na fill
4. Jeśli filled → SUCCESS (maker rebate!)
5. Jeśli nie filled → Cancel i MARKET order (fallback)

Fee Tracking

```python
# Pobierz statystyki
stats = executor.get_fee_stats()
print(f"Maker orders: {stats['total_maker_orders']}")
print(f"Taker orders: {stats['total_taker_orders']}")
print(f"Net fees: ${stats['net_fees']}")
print(f"Maker ratio: {stats['maker_ratio']}%")
```

Dodawanie nowych par

Wymagania

- Minimalne volume: >$10M/day
- Maksymalny spread: <0.1%
- Dostępność na Binance Testnet

Sugerowane pary

Tier	Symbol	Nazwa	Kategoria
Top	SOLUSDC	Solana	L1
Top	ADAUSDC	Cardano	L1
Top	AVAXUSDC	Avalanche	L1
Top	MATICUSDC	Polygon	L2
Mid	LINKUSDC	Chainlink	Oracle
Mid	DOTUSDC	Polkadot	L1
Mid	UNIUSDC	Uniswap	DeFi
Mid	ATOMUSDC	Cosmos	L1
Volatile	SHIBUSDC	Shiba Inu	Meme
Volatile	PEPEUSDC	Pepe	Meme

Jak dodać parę

1. **Edytuj config.yaml:**

```yaml
trading:
  pairs:
    - "BTCUSDC"
    - "ETHUSDC"
    - "SOLUSDC"  # Nowa para
```

1. **Uruchom backtest:**

```
python main.py --backtest --pairs SOLUSDC
```

1. **Walidacja pary:**

```python
from pair_helper import PairHelper

helper = PairHelper(config)
result = helper.validate_pair('SOLUSDC')
print(f"Valid: {result['valid']}")
print(f"Volume: ${result['checks']['volume']:,.0f}")
print(f"Spread: {result['checks']['spread_pct']:.3f}%")
```

Auto-detection

```python
from pair_helper import PairHelper

helper = PairHelper(config)

# Pobierz dostępne pary
pairs = helper.fetch_available_pairs(quote_asset='USDC')

# Filtruj po volume
high_volume = helper.filter_by_volume(pairs, min_volume_usdc=10_000_000)

# Sugestie
suggestions = helper.suggest_pairs(current_pairs=['BTCUSDC', 'ETHUSDC'])
```

API Endpoints

Dashboard Endpoints

Endpoint	Opis
`/api/kelly`	Kelly Criterion stats
`/api/scale_out`	Scale out stats per position
`/api/fees`	Fee tracking summary
`/api/profitability`	Combined profitability metrics

Przykład response `/api/kelly`:

```
{
  "enabled": true,
  "current": {
    "kelly_fraction": 0.15,
    "half_kelly": 0.075,
    "optimal_position_pct": 3.5,
    "win_rate": 65.0,
    "avg_win_rr": 2.1,
    "avg_loss_rr": 1.0,
    "trades_analyzed": 50,
    "confidence": "HIGH",
    "fallback_used": false
  }
}
```

Przykład response `/api/fees`:

```
{
  "total_maker_orders": 150,
  "total_taker_orders": 50,
  "total_fees_paid": 25.0,
  "total_rebates_earned": 15.0,
  "net_fees": 10.0,
  "maker_ratio": 75.0,
  "savings_vs_all_taker": 45.0
}
```

Testy

```
# Uruchom wszystkie testy
python -m pytest tests/ -v

# Testy Kelly
python -m pytest tests/test_kelly.py -v

# Testy Scale Out
python -m pytest tests/test_scale_out.py -v

# Testy Execution
python -m pytest tests/test_execution.py -v
```

Migration Guide

Z v2.0 do v3.0

1. **Dodaj nowe sekcje do config.yaml:**
   - `position_sizing`
   - `profit_taking`
   - `execution`

2. **Backward compatibility:**
   - Wszystkie nowe funkcje są domyślnie włączone
   - Można wyłączyć przez `enabled: false`
   - Fallback do poprzednich metod jeśli wyłączone

3. **Nowe zależności:**
   - Brak nowych zależności (używa istniejących)

Troubleshooting

Kelly zwraca fallback

- Sprawdź czy masz >20 trade'ów w historii
- Upewnij się że trade'y mają `pnl` i `risk_amount`

Scale Out nie triggeruje

- Sprawdź czy `scale_out.enabled: true`
- Upewnij się że pozycja jest zarejestrowana
- Sprawdź czy monitoring jest uruchomiony

Maker orders nie fillują

- Zwiększ `limit_order_timeout`
- Zmniejsz `aggression` (bliżej best bid/ask)
- Sprawdź czy spread jest akceptowalny